

Technical Requirement Document

Here is the Technical Requirement Document (TRD) based on the provided Functional Requirement Document (FRD):

TR-DLT-001: Load Customer and Order Data to Delta Tables

Related Functional Requirement(s): FR-DLT-001

Objective: Implement a Delta Live Table to load customer and order data from CSV files into Delta tables.

Target Cluster Configuration:

- Cluster Name: Gen AI POC Databricks Cluster
- Databricks Runtime Version: 10.4.x-scala2.12
- Node Type: Standard_DS3_v2
- Driver Node: Standard_DS3_v2
- Worker Nodes: 2-4 Standard_DS3_v2
- Autoscaling: Enabled
- Auto Termination: 30 minutes
- Libraries Installed: delta-lake, spark-csv

Source Data Details:

- Dataset Name: Customer Data
- Location: ``/Volumes/gen_ai_poc_databrickscoe/sdlc_wizard/customerdata``
- Format: CSV
- Description: Customer data in CSV format
- Dataset Name: Order Data
- Location: ``/Volumes/gen_ai_poc_databrickscoe/sdlc_wizard/orderdata``
- Format: CSV
- Description: Order data in CSV format

Target Data Details:

- Output Name: Customer Delta Table
- Location: ``gen_ai_poc_databrickscoe.sdlc_wizard.customer``
- Format: Delta
- Description: Customer data in Delta format
- Output Name: Order Delta Table

- Location: ``gen_ai_poc_databrickscoe.sdlc_wizard.order``
- Format: Delta
- Description: Order data in Delta format

Job Flow / Pipeline Stages:

- Read customer CSV data and load it into a Delta table
- Read order CSV data and load it into a Delta table
- Add a new column "TotalAmount" to the "order" table

Data Transformations / Business Logic:

- Step: Load customer data
- Description: Read customer CSV data and load it into a Delta table
- Transformation Logic: Use ``spark.read.csv()`` to read CSV data and ``write.format("delta").saveAsTable()`` to write to Delta table
- Step: Load order data
- Description: Read order CSV data and load it into a Delta table
- Transformation Logic: Use ``spark.read.csv()`` to read CSV data and ``write.format("delta").saveAsTable()`` to write to Delta table
- Step: Calculate TotalAmount
- Description: Add a new column "TotalAmount" to the "order" table
- Transformation Logic: Use ``withColumnn()`` to add a new column and calculate the product of "PricePerUnit" and "Qty"

Error Handling and Logging:

- Log errors to a designated log file
- Use try-except blocks to catch and handle exceptions

TR-DLT-002: Clean and Process Customer and Order Data

Related Functional Requirement(s): FR-DLT-002

Objective: Implement data cleaning and processing logic to remove null and duplicate records from customer and order Delta tables.

- ... (similar details as above)

TR-DLT-003: Create Ordersummary Table and Load Data

Related Functional Requirement(s): FR-DLT-003

Objective: Implement a Delta Live Table to create an "ordersummary" table and load data into it by joining customer and order data.

- ... (similar details as above)

TR-DLT-004: Update Ordersummary Table on Customer Changes

Related Functional Requirement(s): FR-DLT-004

Objective: Implement logic to update the "ordersummary" table whenever there are changes to the customer data.

- ... (similar details as above)

TR-DLT-005: Create Customer Aggregate Spend Table and Load Data

Related Functional Requirement(s): FR-DLT-005

Objective: Implement a Delta Live Table to create a "customeraggregatespend" table and load aggregated data into it.

- ... (similar details as above)

TR-DLT-006: Implement Delta Live Tables

Related Functional Requirement(s): FR-DLT-006

Objective: Implement the above functionality using Delta Live Tables to ensure data is processed and loaded correctly.

- ... (similar details as above)

Here are the detailed pointers for each technical requirement:

TR-DLT-001

Job Flow / Pipeline Stages:

- Read customer CSV data and load it into a Delta table
- Read order CSV data and load it into a Delta table
- Add a new column "TotalAmount" to the "order" table

Data Transformations / Business Logic:

- Step: Load customer data
- Description: Read customer CSV data and load it into a Delta table
- Transformation Logic: Use ``spark.read.csv()`` to read CSV data and ``write.format("delta").saveAsTable()`` to write to Delta table

TR-DLT-002

Job Flow / Pipeline Stages:

- Remove null records from both "customer" and "order" tables
- Remove duplicate records from both "customer" and "order" tables

Data Transformations / Business Logic:

- Step: Remove null records
- Description: Remove null records from both "customer" and "order" tables
- Transformation Logic: Use `dropna()` to remove null records

TR-DLT-003

Job Flow / Pipeline Stages:

- Create "ordersummary" table if it does not exist
- Join "customer" and "order" tables on "CustId" and load the resulting data into the "ordersummary" table

Data Transformations / Business Logic:

- Step: Join customer and order data
- Description: Join "customer" and "order" tables on "CustId"
- Transformation Logic: Use `join()` to join the two tables on "CustId"

TR-DLT-004

Job Flow / Pipeline Stages:

- Identify changes to customer data
- Update "ordersummary" table by making old records inactive and new records active

Data Transformations / Business Logic:

- Step: Identify changes to customer data
- Description: Identify changes to customer data
- Transformation Logic: Use `exceptAll()` to identify changes to customer data

TR-DLT-005

Job Flow / Pipeline Stages:

- Create "customeraggregatespend" table if it does not exist
- Aggregate "TotalAmount" from "ordersummary" table by "Name" and "Date"

Data Transformations / Business Logic:

- Step: Aggregate TotalAmount

- Description: Aggregate "TotalAmount" from "ordersummary" table by "Name" and "Date"
- Transformation Logic: Use `groupBy()` and `sum()` to aggregate "TotalAmount"

TR-DLT-006

Job Flow / Pipeline Stages:

- Implement Delta Live Tables to ensure data is processed and loaded correctly

Data Transformations / Business Logic:

- Step: Implement Delta Live Tables
- Description: Implement Delta Live Tables to ensure data is processed and loaded correctly
- Transformation Logic: Use Delta Live Tables to implement the above functionality